

Operating Systems

Project Report

Implementation & Simulation of CPU Scheduling • Memory Allocation • Page Replacement

T E A M M E M B E R S

#	Full Name	Student ID
1	Shaymaa Mahmoud El Azazy	2305265
2	Haidy Mohamed Anter	2305309
3	Rodaina Mohamed Farag	2305287
4	Hamza Mohamed Ahmed Zayton	2305486
5	Yehia Zakaria Lotfy	2405071

1. Introduction

An Operating System (OS) serves as the fundamental layer of software that manages all hardware resources and provides a stable, consistent environment in which application programs can run. Among the most critical responsibilities of any modern operating system are the management of CPU time, the allocation of primary memory, and the handling of virtual memory through paging mechanisms. These three domains are deeply interrelated, and together they determine the performance, responsiveness, and efficiency of a computing system.

CPU scheduling is the mechanism by which the operating system determines which process in the ready queue should be granted access to the CPU at any given moment. Since most systems are required to support multiple concurrent processes, an effective scheduling algorithm must balance competing objectives such as maximizing CPU utilization, minimizing waiting time, ensuring fairness across processes, and meeting real-time deadline requirements. The scheduler's decisions affect every aspect of system responsiveness and throughput.

Memory management is equally essential. When multiple processes are loaded into RAM simultaneously, the operating system must allocate memory blocks to each process without conflict, track which regions are free or occupied, and minimize wasted space. Different allocation strategies such as First Fit, Best Fit, and Worst Fit each offer distinct trade-offs between speed of allocation and the degree of fragmentation introduced over time.

Virtual memory and page replacement extend the concept of memory management by allowing processes to use more address space than is physically available in RAM. When physical memory is full and a new page must be loaded, the OS must select a victim page to evict. Page replacement algorithms such as LRU, FIFO, and Optimal differ substantially in their ability to minimize page faults and reduce costly disk I/O.

This project implements and simulates three of these algorithms: Round Robin for CPU scheduling, First Fit (with block splitting) for memory allocation, and LRU for page replacement. The goal is to observe their behaviour in practice, visualize their outputs, and draw meaningful comparisons with alternative algorithms. The following sections present each algorithm in detail alongside the actual simulation outputs.

2. CPU Scheduling: Round Robin

2.1 Theoretical Background

Round Robin (RR) is one of the oldest, simplest, and most widely used CPU scheduling algorithms. It was designed specifically for time-sharing systems, in which multiple users or processes are expected to receive prompt and equitable access to the CPU. The defining characteristic of Round Robin is the use of a fixed time quantum (also called a time slice), which represents the maximum amount of CPU time that any process may hold in a single turn before being preempted and returned to the end of the ready queue.

The concept of a time quantum is central to understanding the trade-offs in Round Robin scheduling. A very small quantum produces a system that is highly responsive and fair, but it also causes frequent context switches, each of which imposes overhead as the OS must save and restore CPU register states, update process control blocks, and flush or reload cache contents. Conversely, a very large quantum degrades responsiveness and, in the extreme case where the quantum exceeds all burst times, Round Robin degenerates into First Come First Served (FCFS) scheduling.

Fairness is a defining property of Round Robin. Unlike priority-based schedulers that may starve low-priority processes, or FCFS which may force short processes to wait behind long ones (the convoy effect), Round Robin guarantees that every process in the ready queue will receive CPU time within at most $(n-1) \times \text{quantum}$ time units. This makes it especially suitable for interactive systems and real-time environments where response time is critical.

2.2 Algorithm Step-by-Step

- All processes are placed in a circular ready queue ordered by arrival time.
- The CPU dispatcher picks the first process and runs it for one time quantum (or until completion).
- If the process completes within its quantum, it exits and its completion time is recorded.
- If the process does not complete, it is preempted and added to the tail of the ready queue.
- Any newly arrived processes are enqueued before the preempted process is re-queued.
- This cycle continues until all processes have completed.
- Waiting time and turnaround time are computed after scheduling is complete.

2.3 Simulation Input

Process	Arrival Time	Burst Time	Time Quantum
P1	0	5	2
P2	1	4	2
P3	2	2	2
P4	3	1	2

2.4 Simulation Output — Gantt Chart

The Gantt chart below is the actual output produced by the simulation. The execution sequence is: P1 (0–2), P2 (2–4), P3 (4–6), P1 (6–8), P4 (8–9), P2 (9–11), P1 (11–12).

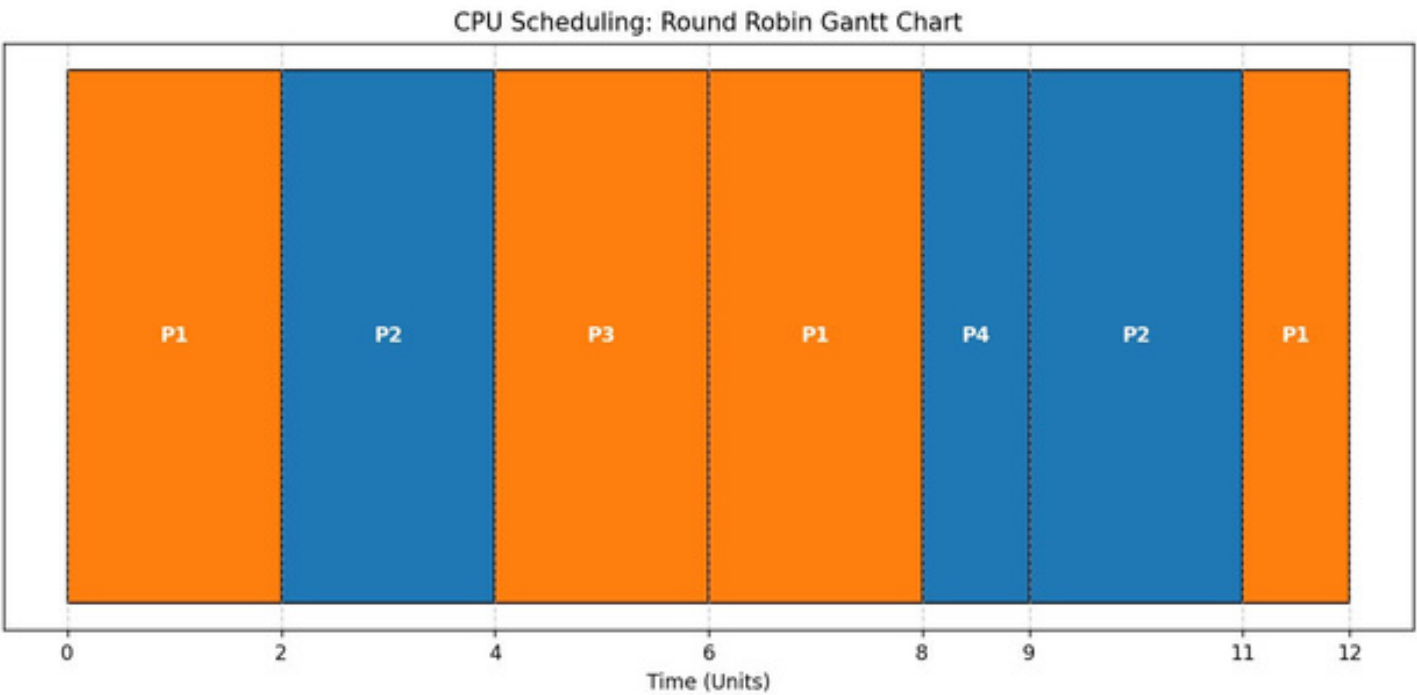


Figure 1: Round Robin Gantt Chart (Time Quantum = 2)

Analysing the Gantt chart step by step: P1 arrives first (t=0) and runs until t=2. P2 runs t=2–4. P3 runs t=4–6 and completes. P1 resumes t=6–8. P4 runs t=8–9 and completes. P2 resumes t=9–11 and completes. P1 takes its final unit t=11–12. The performance metrics are:

Process	Arrival	Burst	Completion	Turnaround Time	Waiting Time
P1	0	5	12	12	7
P2	1	4	11	10	6
P3	2	2	6	4	2
P4	3	1	9	6	5

Average Waiting Time = $(7+6+2+5)/4 = 5.0$ time units. Average Turnaround Time = $(12+10+4+6)/4 = 8.0$ time units. These values reflect the inherent trade-off in Round Robin: while no process waits indefinitely, processes with larger burst times accumulate significant waiting time. Nevertheless, the system is demonstrably fair and starvation-free — the primary design goal of Round Robin scheduling.

3. Memory Allocation: First Fit

3.1 Theoretical Background

Memory allocation is the process by which the operating system assigns portions of available physical memory to processes that request it. The First Fit strategy is the simplest and typically the fastest approach: it scans the free block list from the beginning and allocates the very first block that is large enough to satisfy the request. Because it stops searching immediately upon finding a viable candidate, it minimizes search time.

Block splitting is applied when a selected free block is larger than the process's request. Rather than allocating the entire block (which would waste the surplus), the allocator splits it into two parts: one exactly the size needed by the process, and the remainder returned to the free list. This significantly improves memory utilization and reduces internal fragmentation.

Fragmentation is a critical concern. Internal fragmentation occurs when an allocated block is larger than needed, wasting space inside the allocation. External fragmentation occurs when total free memory is sufficient to satisfy a request, but it is scattered in non-contiguous pieces — no single block is large enough. First Fit with splitting reduces internal fragmentation substantially but does not eliminate external fragmentation.

3.2 Algorithm Step-by-Step

- Initialize the memory with a list of free blocks of given sizes.
- For each incoming process request, scan the free block list from the beginning.
- For each free block, check if its size $\geq$ the process size.
- If suitable: allocate. If block is larger, split it and return the remainder to the free list.
- If no block is large enough, the process fails allocation.
- Continue until all processes have been considered.

3.3 Simulation Input

Block	Size (KB)
B1	100
B2	500
B3	200
B4	300
B5	600

Process	Requested Size (KB)
P1	212
P2	417
P3	112
P4	426

3.4 Simulation Output — Memory Visualization

The chart below is the actual simulation output. Green bars represent free blocks or free fragments; red bars represent blocks allocated to processes.

Simulation Output — dynamic Memory Visualization

Memory Layout: Dynamic (Splitting)

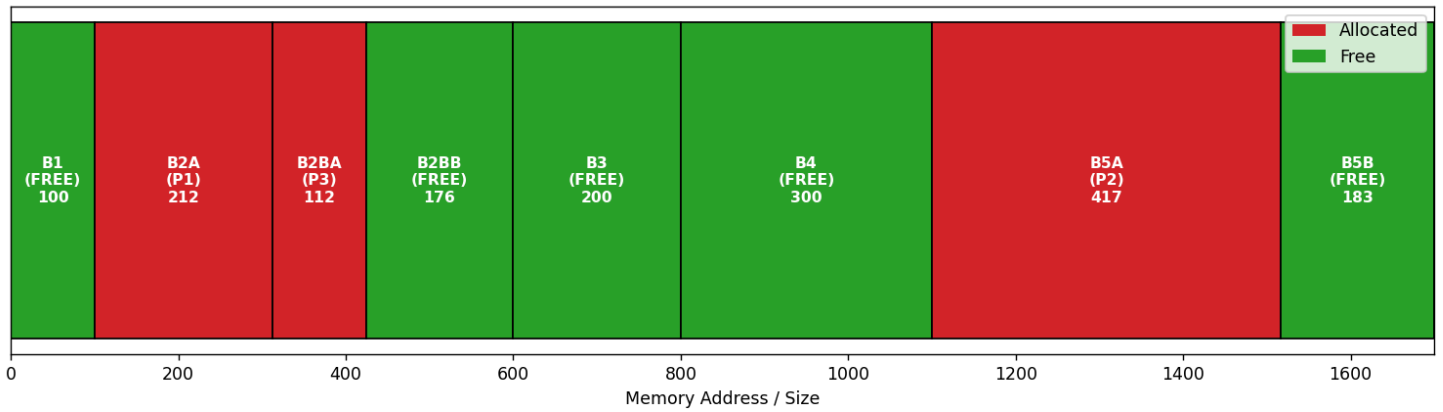


Figure 2: First Fit Memory Allocation Visualization (with Splitting)

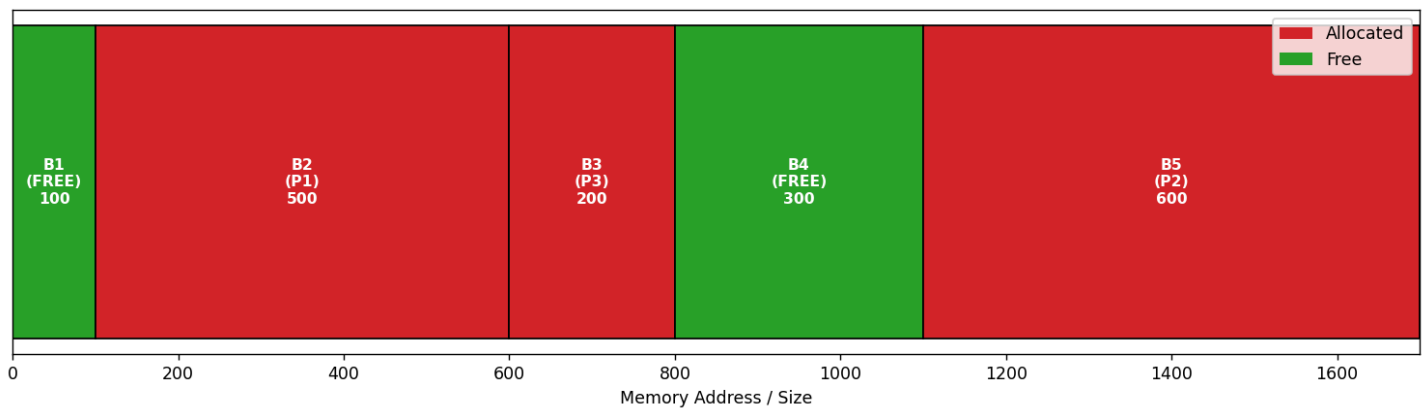
The allocation proceeds as follows: P1 (212 KB) — B1 too small (100 KB), B2 sufficient (500 KB). Allocate 212 KB, split remainder 288 KB back as free. P2 (417 KB) — scans B1, B2 fragment, B3, B4 — all too small. B5 (600 KB) sufficient. Allocate 417 KB, remainder 183 KB returned. P3 (112 KB) — B1 too small, B2 fragment (288 KB) sufficient. Allocate 112 KB, remainder 176 KB. P4 (426 KB) — scans all blocks: B1 (100), B2 fragment (176), B3 (200), B4 (300), B5 fragment (183) — none is large enough. P4 FAILS.

Total free memory = 959 KB, yet P4 cannot be served — a classic demonstration of external fragmentation .

Simulation Output — Static Memory Visualization

The chart below shows the simulation output using Static Partitioning (No Splitting). In this scheme, once a block is assigned to a process, the entire block is occupied, regardless of whether the process needs all of it.

Memory Layout: Static (No Splitting)



P1 (212 KB): Scans blocks. B1 (100 KB) is too small. B2 (500 KB) is sufficient. The entire 500 KB of B2 is allocated to P1. This results in 288 KB of Internal Fragmentation within B2.

P2 (417 KB): Scans remaining free blocks. B1 (100 KB), B3 (200 KB), and B4 (300 KB) are too small. B5 (600 KB) is sufficient. The entire 600 KB of B5 is allocated to

P2. This results in 183 KB of Internal Fragmentation within B5.

P3 (112 KB): Scans remaining free blocks. B1 (100 KB) is too small. B3 (200 KB) is sufficient. The entire 200 KB of B3 is allocated to P3. This results in 88 KB of Internal Fragmentation within B3.

P4 (426 KB): Scans remaining free blocks. B1 (100 KB) and B4 (300 KB) are the only free blocks left. Neither is large enough. P4 FAILS to allocate.

total internal fragmentation is $288 + 183 + 88 = 559$ KB

The Total External Fragmentation is 400 KB

4. Page Replacement: LRU

4.1 Theoretical Background

Virtual memory allows the OS to present each process with the illusion of a large, private address space, even when physical RAM is much smaller. When all physical frames are occupied and a new page must be loaded, the OS must select a victim page to evict. The Least Recently Used (LRU) algorithm evicts the page whose most recent access was furthest in the past, leveraging the principle of temporal locality: a page not used for a long time is unlikely to be needed soon.

LRU provides a much closer approximation to the theoretical Optimal (OPT) algorithm than FIFO does, consistently achieving lower page fault rates for workloads with temporal and spatial locality. Its primary disadvantage is implementation cost: maintaining exact access-time ordering requires hardware counters or software data structures (stacks, linked lists with hash maps) that add overhead on every memory access.

4.2 Simulation Input

Parameter	Value
Number of Frames	3
Reference String	7 0 1 2 0 3 0 4 2 3 0 3 2
Total References	13

4.3 Simulation Output — Frame Evolution

The frame evolution table below is the actual output of the simulation. Red cells indicate a page fault; blue cells indicate a page hit.

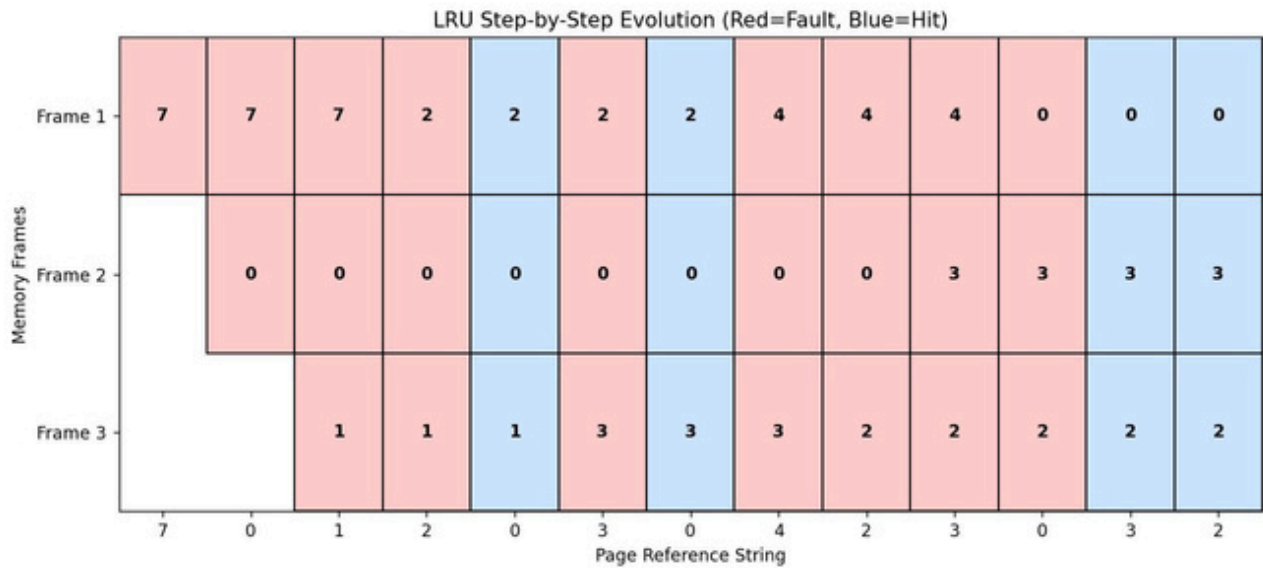


Figure 3: LRU Step-by-Step Frame Evolution (Red = Fault, Blue = Hit)

Step-by-step trace: Ref 7 → FAULT, load into F1. Ref 0 → FAULT, load F2. Ref 1 → FAULT, load F3. Ref 2 → FAULT, evict LRU (page 7), load 2. Ref 0 → HIT. Ref 3 → FAULT, evict page 1 (LRU), load 3. Ref 0 → HIT. Ref 4 → FAULT, evict page 2 (LRU), load 4. Ref 2 → FAULT, evict page 3 (LRU), load 2. Ref 3 → FAULT, evict page 4 (LRU), load 3. Ref 0 → HIT. Ref 3 → HIT. Ref 2 → HIT.

4.4 Simulation Output — Performance Summary

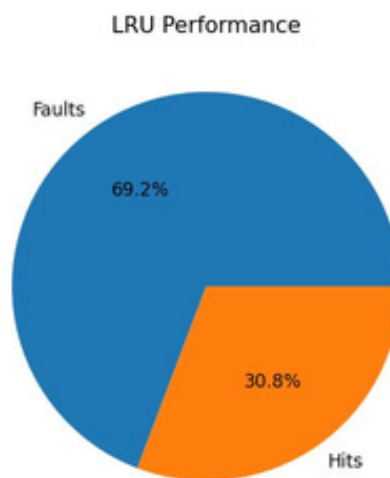


Figure 4: LRU Performance — Hits vs Faults (Pie Chart)

Total References = 13. Page Faults = 9 (69.2%). Page Hits = 4 (30.8%). The first three faults are compulsory cold-start misses, unavoidable regardless of algorithm. The high fault rate reflects the small frame count (3) relative to the working set size (8 distinct pages). The four consecutive hits at the end (references 10–13) confirm that LRU correctly captured the active working set $\{0, 2, 3\}$ in its three frames by the final stage.

5. Comparison and Analysis

5.1 Round Robin vs. First Come First Served (FCFS)

FCFS executes processes in exact arrival order without preemption. While easy to implement and starvation-free, it suffers from the convoy effect, where a long-burst process forces all subsequent processes to wait, dramatically increasing average waiting time. Round Robin addresses this by preempting processes after each quantum, bounding the response time to $n \times \text{quantum}$. The cost is frequent context switches, which add overhead.

Criterion	Round Robin	FCFS
Preemption	Yes (at quantum expiry)	No (non-preemptive)
Fairness	High — all processes served cyclically	Low — convoy effect possible
Response Time	Short (bounded by $n \times \text{quantum}$)	Unpredictable
Context Switches	Frequent	Minimal
Starvation	Never	Never
Best Use Case	Interactive / Time-sharing OS	Batch / Non-interactive

FCFS Simulation Results: While Round Robin (RR) ensures fairness via preemption, First-Come, First-Served (FCFS) executes processes in their exact arrival order. Using the project’s process data (P1:5,P2:4,P3:2,P4:1), the FCFS execution sequence is: P1 (0–5) → P2 (5–9) → P3 (9–11) → P4 (11–12).

Analysis: FCFS shows slightly lower averages here because it avoids the

overhead of context switching. However, RR is the superior choice for Interactive Systems because it prevents the “Convoy Effect” where short processes are stuck behind long-burst processes.

Metric	Round Robin (q=2)	FCFS (Alternative)
Avg. Waiting Time	5.0	4.75
Avg. Turnaround Time	8.0	7.75

5.2 First Fit vs. Best Fit

Best Fit searches the entire free block list and selects the smallest block still large enough to satisfy the request, minimizing wasted space per allocation. However, it tends to leave very small unusable fragments throughout memory. First Fit allocates the first adequate block, leaving a potentially larger, reusable remainder. Empirical studies show First Fit achieves comparable or better overall utilization while being significantly faster.

Criterion	First Fit	Best Fit
Search Strategy	Stops at first suitable block	Scans entire free list
Speed	Faster	Slower
Internal Fragmentation	Moderate	Lower
External Fragmentation	Moderate	Higher (tiny fragments)
Best Use Case	General-purpose OS	Space-constrained systems

Best Fit Simulation Results: Using this memory blocks (100,500,200,300,600) and processes (212,417,112,426):

Step-by-Step: Best Fit Static Allocation:

Using blocks: B1(100), B2(500), B3(200), B4(300), B5(600).

1. P1 (212 KB):
 - Scan: 100 (No), 500 (Yes), 200 (No), 300 (Yes), 600 (Yes).
 - Selection: B4 (300 KB) is the smallest that fits.
 - Result: P1 is placed in B4. Internal Fragmentation = 88 KB.
2. P2 (417 KB):
 - Scan: 100 (No), 500 (Yes), 200 (No), 600 (Yes).
 - Selection: B2 (500 KB) is the smallest that fits.
 - Result: P2 is placed in B2. Internal Fragmentation = 83 KB.
3. P3 (112 KB):
 - Scan: 100 (No), 200 (Yes), 600 (Yes).
 - Selection: B3 (200 KB) is the smallest that fits.
 - Result: P3 is placed in B3. Internal Fragmentation = 88 KB.
4. P4 (426 KB):
 - Scan: 100 (No), 600 (Yes).
 - Selection: B5 (600 KB).
 - Result: P4 is placed in B5. Internal Fragmentation = 174 KB.

Total Internal Fragmentation (Best Fit Static): $88 + 83 + 88 + 174 = 433\text{KB}$

Total External Fragmentation (Best Fit Static): Only **B1 (100 KB)** remains completely empty and unused.

While Best Fit is theoretically known for creating small fragments, in this specific simulation, it performed significantly better than First Fit. It reduced total waste by **426 KB**

5.3 LRU vs. First In First Out (FIFO)

FIFO maintains a queue of pages in load order, always evicting the oldest-loaded page regardless of whether it is still actively used. It makes no use of access frequency information and is susceptible to Belady's Anomaly (more frames can increase page faults). LRU leverages temporal locality to evict the least recently accessed page, consistently achieving lower fault rates and immunity to Belady's Anomaly, at the cost of higher implementation complexity.

Criterion	LRU	FIFO
Eviction Policy	Least recently accessed	Oldest loaded page
Access Info Used	Yes — full access history	No — only load order
Page Fault Rate	Lower — closer to optimal	Higher
Belady's Anomaly	Not susceptible	Susceptible
Implementation Cost	Higher (timestamps / stack)	Low (simple queue)
Best Use Case	General-purpose, locality-heavy	Simple embedded systems

FIFO Simulation Results: for the 13- reference string. Using the same reference string (7,0,1,2,0,3,0,4,2,3,0,3,2) with 3 frames:

FIFO Page Faults: 10

LRU Page Faults: 9

Comparison :

While FIFO is simpler to implement, it resulted in **10 Page Faults**, compared to **9** for LRU. This occurs because FIFO ignores how recently a page was used and simply replaces the oldest one, which occasionally leads to 'Belady's Anomaly' where more frames don't necessarily mean fewer faults. LRU proved more efficient here by keeping frequently accessed pages like '0' and '3' in memory longer.

6. Conclusion

This project provided a comprehensive, hands-on exploration of three foundational operating systems algorithms: Round Robin CPU scheduling, First Fit memory allocation with block splitting, and Least Recently Used page replacement. By implementing these algorithms in software and examining their actual simulation outputs — Gantt charts, memory allocation maps, and frame evolution tables — the team gained deep practical understanding that complements theoretical study.

The Round Robin simulation demonstrated how a fixed time quantum ensures fairness and bounded response time across four processes with different arrival and burst times. The Gantt chart made every scheduling decision transparent and verifiable, confirming that no process was starved while maintaining predictable turnaround times.

The First Fit simulation illustrated both the power and the limitations of dynamic partitioning. Block splitting significantly improved memory utilization by returning unused portions of large blocks to the free pool. However, the failure to allocate P4 despite a theoretically sufficient total free memory (959 KB) vividly demonstrated external fragmentation — a challenge that motivates advanced techniques such as compaction, paging, and buddy system allocation.

The LRU simulation reinforced the importance of temporal locality. Although the reference string produced a 69.2% fault rate with only 3 frames (largely due to the large working set and unavoidable cold-start misses), the algorithm correctly retained the active working set in the final steps, achieving four consecutive hits. The comparison with FIFO highlighted LRU's immunity to Belady's Anomaly and its superior exploitation of access patterns.

The comparative analysis reinforced that there is no universally optimal algorithm. Each represents specific design trade-offs between speed, fairness, utilization, and implementation complexity. Understanding these trade-offs is essential for OS designers, system programmers, and computer scientists who must select and tune algorithms for specific workloads and hardware environments. Mastery of these concepts is foundational to careers in systems programming, cloud infrastructure, and computer architecture.

